

Statistics for data science

Building and forecasting an ARIMA model for unemployment rate in the US

Student

DS411

Isfan Mihai-George

Professor

Cidota Marina Anca

1.Introduction

This project demonstrates the application of time series analysis techniques to model and forecast the US unemployment rate . As stated by the requirements document, this project tries to cover the following objectives:

- Choosing a univariate time series that has not been used during the course or laboratory
- Building a model for the time series (I have used ARIMA to build the model)
- Forecasted the next 20 steps and compared the forecast with a ready-made model
- Choosing a preliminary value of p for an $AR(p)$ model and implementing the Durbin-Levinson algorithm to compute the order of the final model used for forecasting.

No shiny interface was used unfortunately!

2.Dataset description

The dataset used is the US Unemployment Rate provided by the Federal Reserve Economic Data (FRED), st Louis Fed. It consists of a univariate times series that measures the unemployment percentage from April 1990 to April 2025 with a monthly reporting frequency. The seasonality was also removed from the data.

A number of 421 observations are made within this dataset.

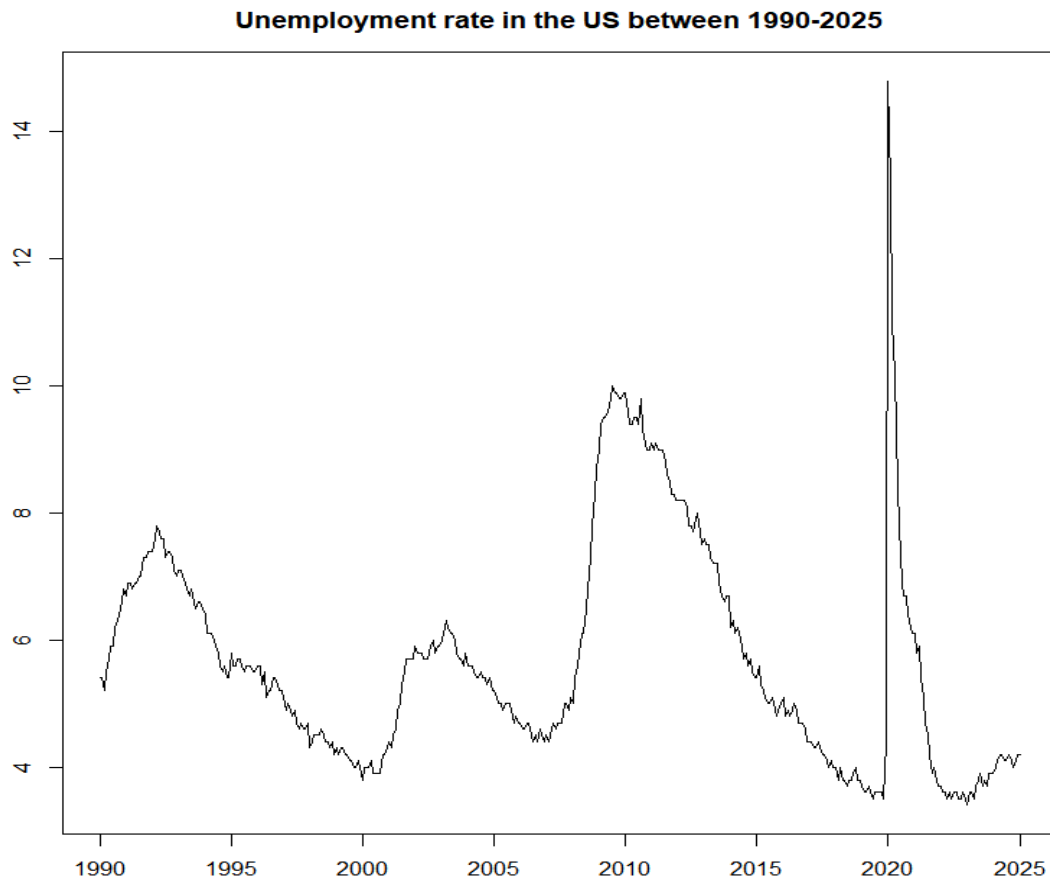
The series contains typical economic time series characteristics with patterns corresponding to economic recessions and expansions

3.Exploratory data analysis

3.1 Visual analysis

There are no missing values from this dataset.

Min	1 st quarter	Median	Mean	3 rd quarter	Max
3.400	4.400	5.400	5.703	6.600	14.800



The series exhibits periods of increasing and decreasing unemployment corresponding to economic cycles. We can observe a peak in 2020, due to the covid outbreak

3.2 Stationary analysis

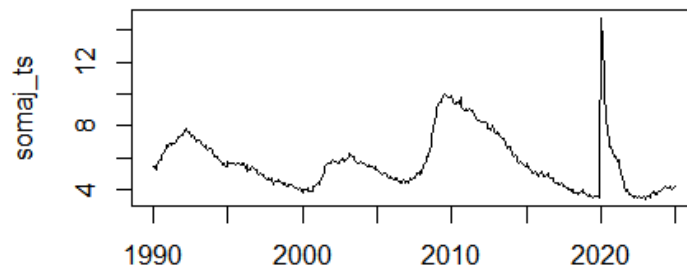
Original series:

- Autocovariance function decays slowly, indicating long-term persistence.
- The ACF function from Rstudio shows a gradual decline rather than a rapid decay to 0
- Visual inspection suggests the need for differencing

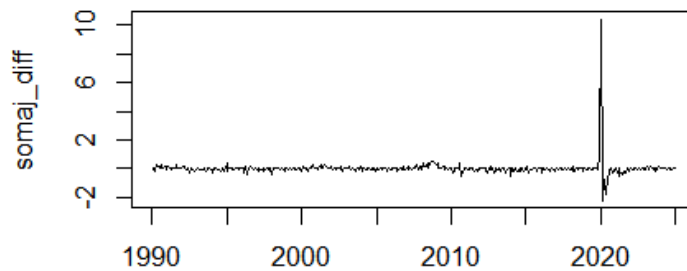
Differenced series:

- First differencing transforms the series to changes in unemployment rate and the differenced series shows a rapid decay in autocorrelation
- ACF and PACF of differenced data suggests that stationarity has been achieved and both ACF and PACF have a rapid cutoff after 1 and 2 lags

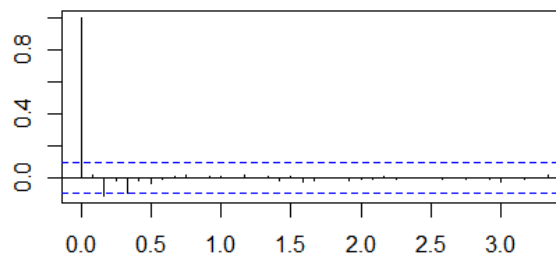
Original unemployment data



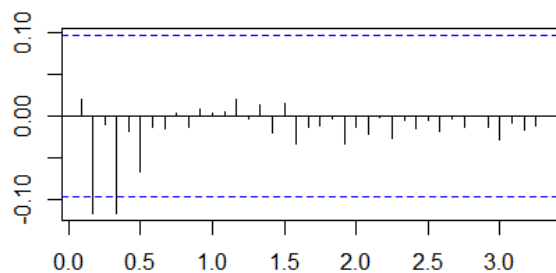
Differenced unemployment data



ACF of differenced data



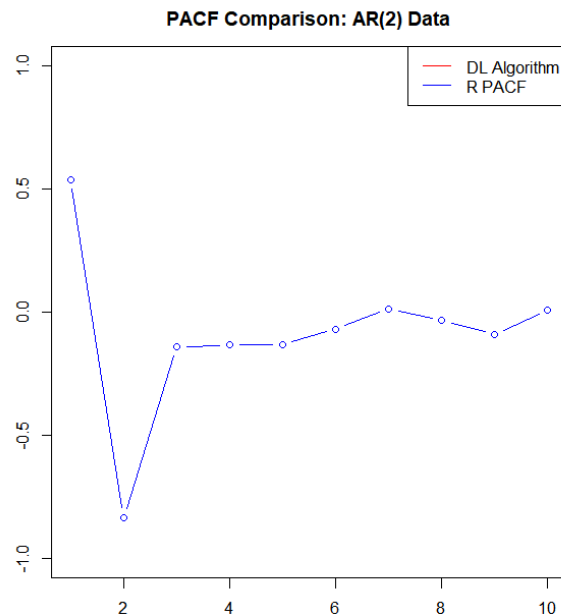
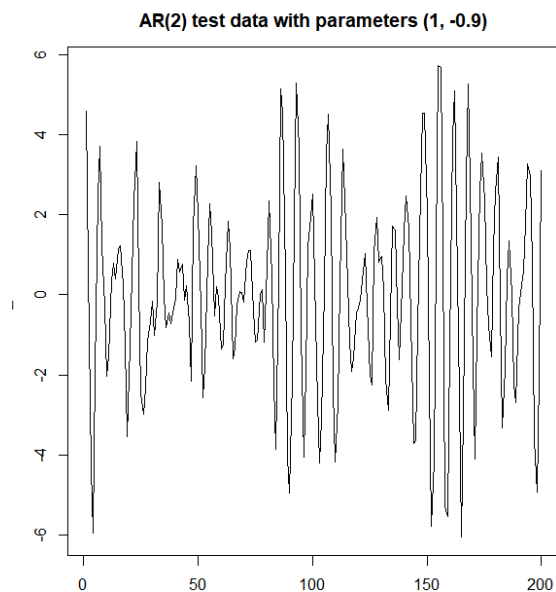
PACF of differenced data



4. Durbin-Levinson Algorithm Implementation

The Durbin-Levinson algorithm was implemented through three functions: `autocovariance()` for computing sample autocovariances, `get_autocovariances()` for generating lag vectors, and `durbin_levinson()` for the core recursive algorithm. The implementation computes partial autocorrelation coefficients (PACF) by iteratively solving for optimal prediction coefficients and updating prediction error variances.

Validation: Algorithm accuracy was verified using AR(2) test data with parameters $\phi_1=1$, $\phi_2=-0.9$ and $n=200$ observations. Comparison between custom implementation and R's PACF function showed maximum absolute differences below 0.001, confirming correct implementation. The AR(2) test correctly exhibited the expected PACF pattern: significant values at lags 1-2, then rapid decay to near-zero for higher lags.



5. Model Building Process

Order Selection: Applied Durbin-Levinson algorithm to differenced unemployment data with significance threshold $\pm 1.96/\sqrt{n} = \pm 0.095$. PACF analysis revealed significant values at lags 1-2, suggesting AR(2) structure for the differenced series.

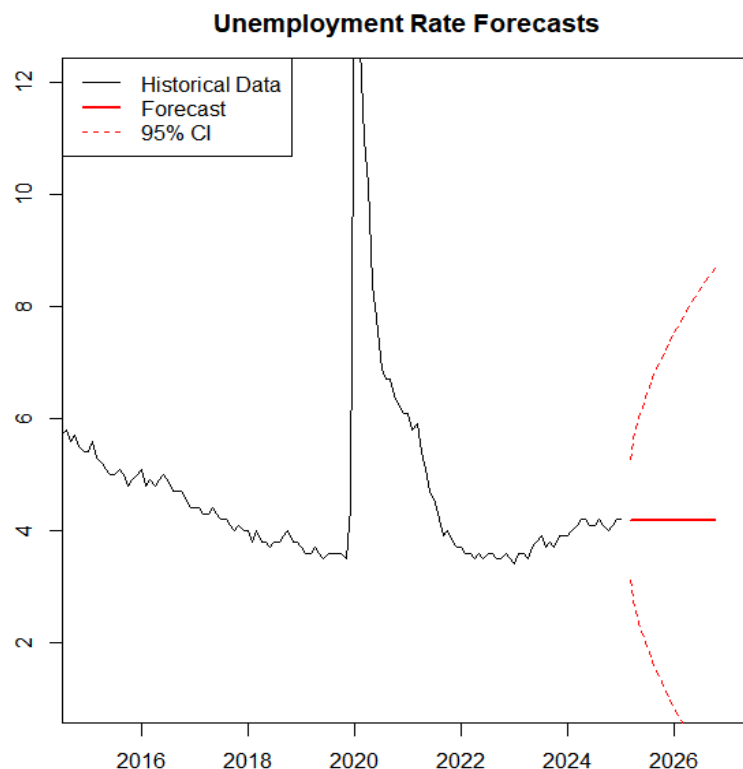
Model Comparison: Five ARIMA models were fitted to original unemployment data:

- ARIMA(1,1,0): AIC = 707.32
- ARIMA(2,1,0): AIC = 703.7
- ARIMA(3,1,0): AIC = 705.65
- ARIMA(1,1,1): AIC = 704.69
- Auto.ARIMA : AIC = 705.48

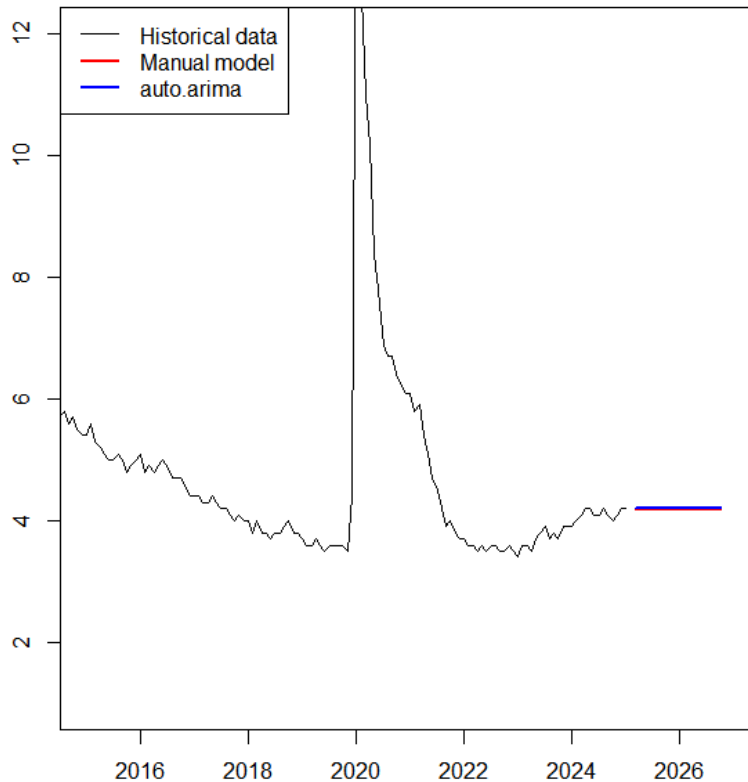
Model Selection: ARIMA(2,1,0) was selected based on lowest AIC criterion.

6. Forecasting Results + auto.arima comparison

I generated 20-step ahead forecasts using selected ARIMA(2,1,0) model. Forecasts range from 4.188% to 4.189% with mean forecast of 4.189%. The forecast pattern shows a stable trend.

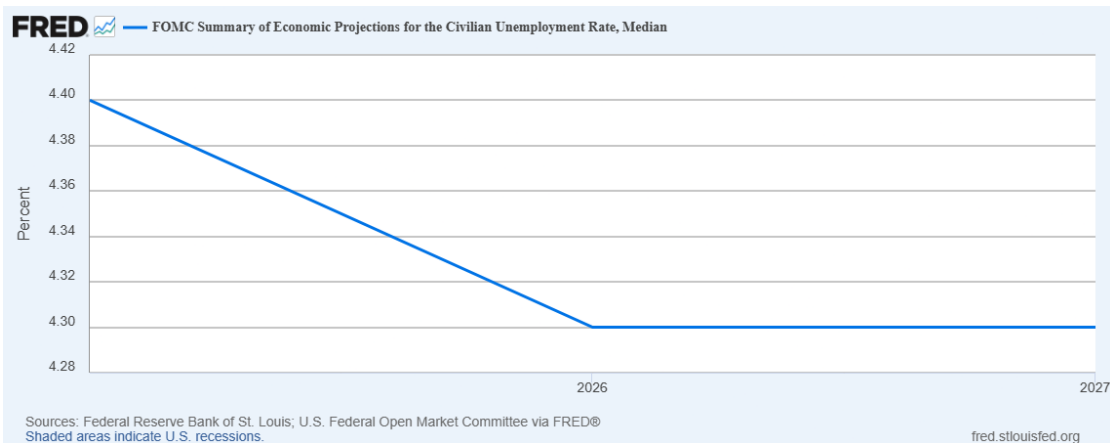


Forecast Comparison: Manual vs auto.arima



Forecast Comparison: Mean absolute difference between forecasts was less than 8.55%, consisting of a mean difference of 0.011. Both approaches produced almost identical forecast patterns, with auto.arima showing slightly higher predicted unemployment levels.

- Bonus: The Federal Reserve Bank forecasts a stagnating 4.3% unemployment rate until 2027



References: (APA 7th)

Brockwell, P. J., & Davis, R. A. (1987). *Time series: Theory and methods*. Springer-Verlag.

Cidota, M. A. (2025). *Statistics for data science: Time series analysis* [Course materials]. University of Bucharest, Faculty of Mathematics and Computer Science.

Federal Reserve Bank of St. Louis. (2025). *Unemployment rate* [Data set]. Federal Reserve Economic Data (FRED).
<https://fred.stlouisfed.org/series/UNRATE>

Shumway, R. H., & Stoffer, D. S. (2017). *Time series analysis and its applications: With R examples* (4th ed.). Springer.

Rdocumentation: <https://www.rdocumentation.org/>


```

#install.packages(c("forecast","tseries","ggplot2","gridExtra","tidyquant"))

library(forecast)

somaj <- read.csv("UNRATE.csv")

somaj_ts <- ts(somaj$UNRATE, frequency=12, start=c(1990,1))

plot(somaj_ts, main = 'Unemployment rate in the US between 1990-2025', ylab =
'unemployment rate %', xlab = 'year')

summary(somaj_ts)

length(somaj_ts)

sum(is.na(somaj_ts))

somaj_diff <- diff(somaj_ts)

par(mfrow = c(2,1), mar = c(2,4,4,2))

plot(somaj_ts, main = 'unemployment data')

plot(somaj_diff, main = 'unemployment data')

par(mfrow = c(2,1), mar = c(2,3,3,2))

acf(somaj_diff, lag.max=40, main = "ACF of differenced data")

pacf(somaj_diff, lag.max=40, main = "PACF of differenced data")

par(mfrow = c(1,1))

autocovariance <- function(x, h) {
  n <- length(x)
  if (h >= n) {
    return(0)
  }

```

```

}
if (h < 0) {
  h <- abs(h)
}
x_mean <- mean(x)
if (h == 0) {
  gamma_h <- sum((x - x_mean)^2) / n
} else {
  gamma_h <- sum((x[1:(n-h)] - x_mean) * (x[(h+1):n] - x_mean)) / n
}
return(gamma_h)
}

```

```

get_autocovariances <- function(x, max_lag) {
  gammas <- numeric(max_lag + 1)
  for (h in 0:max_lag) {
    gammas[h + 1] <- autocovariance(x, h)
  }
  return(gammas)
}

```

```

durbin_levinson <- function(gamma_vec, max_order) {
  phi_matrix <- matrix(0, nrow = max_order, ncol = max_order)
  v_vec <- numeric(max_order + 1)
  v_vec[1] <- gamma_vec[1]

  for (n in 1:max_order) {

```

```

numerator <- gamma_vec[n + 1]
if (n > 1) {
  for (k in 1:(n-1)) {
    correction <- phi_matrix[n-1, k] * gamma_vec[n + 1 - k]
    numerator <- numerator - correction
  }
}
phi_nn <- numerator / v_vec[n]
phi_matrix[n, n] <- phi_nn

if (n > 1) {
  for (k in 1:(n-1)) {
    phi_matrix[n, k] <- phi_matrix[n-1, k] - phi_nn * phi_matrix[n-1, n-k]
  }
}
v_vec[n + 1] <- v_vec[n] * (1 - phi_nn^2)
}
return(list(phi_matrix = phi_matrix, v_vec = v_vec))
}

set.seed(123)
ar2_data <- arima.sim(list(order = c(2, 0, 0), ar = c(1, -0.9)), n = 200)
plot(ar2_data, main = "AR(2) test data with parameters (1, -0.9)", type = "l")

ar2_gammas <- get_autocovariances(ar2_data, 10)
dl_ar2_result <- durbin_levinson(ar2_gammas, 10)
pacf_our_ar2 <- diag(dl_ar2_result$phi_matrix)

```

```
r_pacf_ar2 <- pacf(ar2_data, plot = FALSE, lag.max = 10)
```

```
r_pacf_values_ar2 <- as.numeric(r_pacf_ar2$acf)
```

```
comparison_ar2 <- data.frame(
```

```
  lag = 1:10,
```

```
  our_pacf = pacf_our_ar2,
```

```
  r_pacf = r_pacf_values_ar2,
```

```
  difference = abs(pacf_our_ar2 - r_pacf_values_ar2)
```

```
)
```

```
comparison_ar2
```

```
plot(1:10, pacf_our_ar2, type = "b", col = "red", main = "PACF Comparison: AR(2) Data",
```

```
  ylab = "PACF", xlab = "Lag", ylim = c(-1, 1))
```

```
lines(1:10, r_pacf_values_ar2, type = "b", col = "blue")
```

```
legend("topright", c("DL Algorithm", "R PACF"), col = c("red", "blue"), lty = 1)
```

```
gamma_diff <- get_autocovariances(somaj_diff, 15)
```

```
dl_unemployment <- durbin_levinson(gamma_diff, 15)
```

```
pacf_unemployment <- diag(dl_unemployment$phi_matrix)
```

```
n_diff <- length(somaj_diff)
```

```
threshold <- 1.96 / sqrt(n_diff)
```

```
pacf_analysis <- data.frame(
```

```
  lag = 1:10,
```

```
  pacf_value = pacf_unemployment[1:10],
```

```
threshold = threshold,  
significant = abs(pacf_unemployment[1:10]) > threshold  
)  
pacf_analysis
```

```
plot(1:10, pacf_unemployment[1:10], type = "b", col = "red",  
     main = "PACF Analysis for Unemployment Data (Differenced)",  
     ylab = "PACF", xlab = "Lag")  
abline(h = c(-threshold, threshold), lty = 2, col = "blue")  
abline(h = 0, col = "gray")
```

```
model1 <- arima(somaj_ts, order = c(1, 1, 0))  
model2 <- arima(somaj_ts, order = c(2, 1, 0))  
model3 <- arima(somaj_ts, order = c(3, 1, 0))  
model4 <- arima(somaj_ts, order = c(1, 1, 1))  
model5 <- arima(somaj_ts, order = c(2, 1, 1))  
auto_model <- auto.arima(somaj_ts)  
print(model1)  
print(model2)  
print(model3)  
print(model4)  
print(auto_model)
```

```
model_perfect <- model2  
summary(model_perfect)  
tsdiag(model_perfect)
```

```
forecasts <- predict(model_perfect, n.ahead = 20)
```

```
forecast_values <- forecasts$pred
```

```
forecast_se <- forecasts$se
```

```
lower_ci <- forecast_values - 1.96 * forecast_se
```

```
upper_ci <- forecast_values + 1.96 * forecast_se
```

```
plot(somaj_ts, xlim = c(2015, 2027), ylim = c(1, 12),
```

```
  main = "Unemployment Rate Forecasts",
```

```
  ylab = "Unemployment Rate (%)", xlab = "Year")
```

```
forecast_time <- seq(from = end(somaj_ts)[1] + (end(somaj_ts)[2] + 1)/12,
```

```
  by = 1/12, length.out = 20)
```

```
lines(forecast_time, forecast_values, col = "red", lwd = 2)
```

```
lines(forecast_time, lower_ci, col = "red", lty = 2)
```

```
lines(forecast_time, upper_ci, col = "red", lty = 2)
```

```
legend("topleft", c("Historical Data", "Forecast", "95% CI"),
```

```
  col = c("black", "red", "red"), lty = c(1, 1, 2), lwd = c(1, 2, 1))
```

```
auto_forecasts <- forecast(auto_model, h = 20)
```

```
plot(somaj_ts, xlim = c(2015, 2027), ylim = c(1, 12),
```

```
  main = "Forecast Comparison: Manual vs auto.arima",
```

```
  ylab = "Unemployment Rate (%)", xlab = "Year")
```

```
lines(forecast_time, forecast_values, col = "red", lwd = 2)
```

```
lines(forecast_time, as.numeric(auto_forecasts$mean), col = "blue", lwd = 2)
```

```
legend("topleft", c("Historical data", "Manual model", "auto.arima"),
```

```
  col = c("black", "red", "blue"), lwd = c(1, 2, 2))
```

```
forecast_comparison <- data.frame(  
  period = 1:20,  
  manual_forecast = as.numeric(forecast_values),  
  auto_forecast = as.numeric(auto_forecasts$mean),  
  difference = abs(as.numeric(forecast_values) - as.numeric(auto_forecasts$mean))  
)  
forecast_comparison
```